Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей

Кафедра Информатики

Дисциплина «Информационные сети. Основы безопасности»

ОТЧЁТ

к лабораторной работе №2

на тему

«ЭЛЕМЕНТЫ КРИПТОГРАФИИ»

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

_____
(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Цель работы 3
2 Ход работы 4
2.1 Шифр Цезаря 4
2.2 Шифр Виженера 4
Заключение 6
Приложение А (обязательное) Листинг программного кода 7

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение базовых принципов криптографии на примере простых алфавитных шифров: шифра Виженера и шифра Цезаря. Задачи включают изучение шифров Цезаря и Виженера, проектирование и реализацию программного средства шифрования и дешифрования текстовых файлов при помощи шифров, перечисленных выше и тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, осуществляющая шифрование и дешифрование текстовых файлов указанными шифрами.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Python*, осуществляющее шифрование и дешифрование текстовых файлов шифрами Цезаря и Виженера.

2.1 Шифр Цезаря

Шифр Цезаря – разновидность шифра подстановки, в котором каждый символ в открытом тексте заменяется символом, находящимся на некоторое постоянное число позиций левее или правее него в алфавите. К примеру при шифровании слова «шифр» с ключом 17 зашифрованный текст будет иметь вид «ищеб». Принцип работы шифра Цезаря наглядно представлен на рисунке 2.1

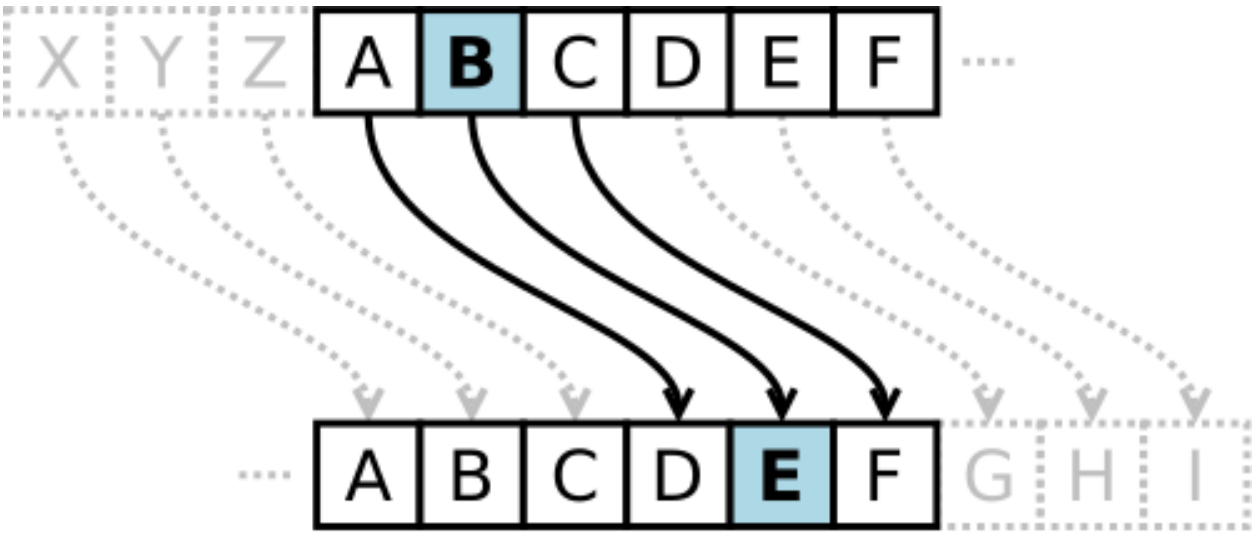

Рисунок 2.1 – Принцип работы шифра Цезаря

Исходный код класса, реализующего шифр Цезаря представлен в листинге А.1.

2.2 Шифр Виженера

Шифр Виженера – разновидность шифра подстановки, в котором каждый символ в открытом тексте заменяется символом, позиция которого в алфавите вычисляется с помощью соответствующей буквы ключа. К примеру при шифровании фразы «посольство Венгрии» ключом «Будапешт» зашифрованный текст будет иметь вид «рвхоыбйегв Ёеэзиый». На рисунке 2.2 представлен квадрат Виженера, позволяющий облегчить ручное шифрование и расшифрование сообщений, зашифрованных шифром Виженера.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Рисунок 2.2 – Квадрат Виженера

Шифр Виженера «размывает» характеристики частотностей появления символов в тексте, но некоторые особенности появления символов в тексте остаются. Главный недостаток шифра Виженера состоит в том, что его ключ повторяется, то есть для исходного текста длины n ключ длины m будет повторён $\lceil \frac{n}{m} \rceil$ раз, последнее из повторений может быть неполным.

Исходный код класса, реализующего шифр Виженера представлен в листинге А.2.

Исходный код функции main, являющейся точкой входа в разработанную программу представлен в листинге А.3.

Таким образом были изучены шифры Цезаря и Виженера, а также была разработана программная реализация данных шифров.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы были изучены и реализованы классические алгоритмы симметричного шифрования — шифр Цезаря и шифр Виженера. На первом этапе была разработана программная логика для шифра Цезаря, основанная на циклическом сдвиге алфавита на фиксированный ключ. Затем был реализован более сложный многоалфавитный шифр Виженера, в котором роль ключа выполняет специальное кодовое слово, генерирующее последовательность сдвигов для каждого символа. Использование ключевой фразы позволило значительно повысить криптостойкость системы за счёт нейтрализации частотного анализа, характерного для простых моноалфавитных подстановок.

После реализации алгоритмов было проведено тестирование на различных наборах данных. С помощью контрольных примеров проверялась корректность процессов зашифрования и расшифрования: текст, прошедший полный цикл преобразований, полностью соответствовал исходному сообщению. Особое внимание было уделено обработке пограничных случаев, таких как использование символов в разных регистрах, пробелов и знаков препинания, что подтвердило стабильность работы алгоритмов в различных условиях.

В результате выполнения работы были успешно освоены основные элементы классической криптографии. Все цели, поставленные перед началом лабораторной работы, были достигнуты: программные модули продемонстрировали полную работоспособность и корректность математических преобразований. Были выполнены ключевые операции по манипуляции алфавитными индексами и работе с циклическими сдвигами. Проверка результатов показала, что реализованные методы обеспечивают надёжную (в рамках данных алгоритмов) передачу зашифрованной информации и её точное восстановление при наличии верного ключа.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Реализация шифра Цезаря на *Python*

```
class Caesar:
    alphabets: list = [
        "abcdefghijklmnopqrstuvwxyz",
        "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"
    ]

    @classmethod
    def __inner_chipher(cls, text: str, key: int, mode: str = "chipher"):
        result = ""
        if mode == "dechipher":
            key *= -1

        for c in text:
            found = False
            for alphabet in cls.alphabets:
                alphabet_lower = alphabet.lower()
                alphabet_upper = alphabet.upper()
                if c in alphabet_lower:
                    idx = alphabet_lower.index(c)
                    new_idx = (idx + key) % len(alphabet_lower)
                    result += alphabet_lower[new_idx]
                    found = True
                    break

                elif c in alphabet_upper:
                    idx = alphabet_upper.index(c)
                    new_idx = (idx + key) % len(alphabet_upper)
                    result += alphabet_upper[new_idx]
                    found = True
                    break

            if not found:
                result += c

        return result

    @classmethod
    def chipher(cls, plaintext: str, key: int) -> str:
        return cls.__inner_chipher(plaintext, key)

    @classmethod
    def dechipher(cls, chiphered_text: str, key: int) -> str:
        return cls.__inner_chipher(chiphered_text, key, "dechipher")

    @classmethod
    def chipher_file(cls, source_file_name: str, key: int, target_file_name:
str | None = None):
        with open(source_file_name, "r") as f:
            content = f.read()

        encrypted_text = cls.chipher(content, key)

        if target_file_name is None:
            p = Path(source_file_name)
            target_file_name = f"{p.stem}_encrypted{p.suffix}"
```

```
        with open(target_file_name, "w") as f:
            f.write(encrypted_text)

    @classmethod
    def dechipher_file(cls, source_file_name: str, key: int,
target_file_name: str | None = None):
        with open(source_file_name, "r") as f:
            content = f.read()

        decrypted_text = cls.dechipher(content, key)

        if target_file_name is None:
            p = Path(source_file_name)
            target_file_name = f"{p.stem}_decrypted{p.suffix}"

        with open(target_file_name, "w") as f:
            f.write(decrypted_text)
```

Листинг А.2 – Реализация шифра Виженера на Python

```
class Vigenere:
    @classmethod
    def chipher(cls, plaintext: str, key: str) -> str:
        return cls.__inner_chipher(plaintext, key)

    @classmethod
    def dechipher(cls, chiphered_text: str, key: str) -> str:
        return cls.__inner_chipher(chiphered_text, key, "dechipher")

    @classmethod
    def __inner_chipher(cls, text: str, key: str, mode: str = "chipher"):
        latin = "abcdefghijklmnopqrstuvwxyz"
        cyrillic = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"

        latin_map = {char: index for index, char in enumerate(latin)}
        cyrillic_map = {char: index for index, char in enumerate(cyrillic)}

        result = []
        key_index = 0
        key = key.lower()

        def get_shift(k_char):
            if k_char in latin_map:
                return latin_map[k_char]
            elif k_char in cyrillic_map:
                return cyrillic_map[k_char]
            return 0

        for char in text:
            lower_char = char.lower()

            if lower_char in latin_map:
                alphabet = latin
                idx = latin_map[lower_char]
                mod = 26
            elif lower_char in cyrillic_map:
                alphabet = cyrillic
                idx = cyrillic_map[lower_char]
                mod = 33
            else:
                result.append(char)
                continue
```

```
            k_char = key[key_index % len(key)]
            shift = get_shift(k_char)

            if mode == 'chipher':
                new_idx = (idx + shift) % mod
            elif mode == 'dechipher':
                new_idx = (idx - shift) % mod
            else:
                raise ValueError("Mode must be 'chipher' or 'dechipher'")

            new_char = alphabet[new_idx]
            if char.isupper():
                new_char = new_char.upper()

            result.append(new_char)

            key_index += 1

        return "".join(result)

    @classmethod
    def chipher_file(cls, source_file_name: str, key: str, target_file_name:
str | None = None):
        with open(source_file_name, "r") as f:
            content = f.read()

        encrypted_text = cls.chipher(content, key)

        if target_file_name is None:
            p = Path(source_file_name)
            target_file_name = f"{p.stem}_encrypted{p.suffix}"

        with open(target_file_name, "w") as f:
            f.write(encrypted_text)

    @classmethod
    def dechipher_file(cls, source_file_name: str, key: str,
target_file_name: str | None = None):
        with open(source_file_name, "r") as f:
            content = f.read()

        decrypted_text = cls.dechipher(content, key)

        if target_file_name is None:
            p = Path(source_file_name)
            target_file_name = f"{p.stem}_decrypted{p.suffix}"

        with open(target_file_name, "w") as f:
            f.write(decrypted_text)
```

Листинг А.3 – Функция, являющаяся точкой входа в программу

```
def main():
    run = True
    while run:
        chipher_type = input("Chipher type ([c]aesar, [v]iginere, [q]uit): ")
        if chipher_type not in ["c", "v", "q"]:
            print("Invalid command")
            continue

        if chipher_type == "q":
```

```
    run = False
    continue

file_name = input("File name: ")
if not os.path.exists(file_name):
    print("Invalid file name")
    continue

mode = input("Mode ([c]ipher, [d]echipher)")
if chipher_type not in ["c", "d"]:
    print("Invalid command")
    continue
chipher: Caesar | Vigenere
if chipher_type == "c":
    chipher = Caesar
    try:
        key = int(input("Key: "))
    except:
        print("Invalid key")
        continue
else:
    chipher = Vigenere
    key = input("Key: ")

if mode == "c":
    chipher.chipher_file(file_name, key)
else:
    chipher.dechipher(file_name, key)

run = False
```